

String Operations

SQL specifies strings by enclosing them in single quotes, for example, 'Computer'. A single quote character that is part of a string can be specified by using two single quote characters; for example, the string “It’s right” can be specified by 'It"s right'.

The SQL standard specifies that the equality operation on strings is case sensitive; as a result, the expression “'comp. sci.' = 'Comp. Sci.'” evaluates to false. However, some database systems, such as MySQL and SQL Server, do not distinguish uppercase from lowercase when matching strings; as a result, “'comp. sci.' = 'Comp. Sci.'” would evaluate to true on these systems. This default behavior can, however, be changed, either at the database level or at the level of specific attributes.

SQL also permits a variety of functions on character strings, such as concatenating (using “||”), extracting substrings, finding the length of strings, converting strings to uppercase (using the function **upper(s)** where *s* is a string) and lowercase (using the function **lower(s)**), removing spaces at the end of the string (using **trim(s)**), and so on. There are variations on the exact set of string functions supported by different database systems. See your database system’s manual for more details on exactly what string functions it supports.

Pattern matching can be performed on strings using the operator like. We describe patterns by using two special characters:

- Percent (%): The % character matches any substring.
- Underscore (_): The _ character matches any character.

Patterns are case sensitive;⁴ that is, uppercase characters do not match lowercase characters, or vice versa. To illustrate pattern matching, we consider the following examples:

- 'Intro%' matches any string beginning with “Intro”.
- '%Comp%' matches any string containing “Comp” as a substring, for example, 'Intro. to Computer Science', and 'Computational Biology'.
- '___' matches any string of exactly three characters.
- '___%' matches any string of at least three characters.

⁴Except for MySQL, or with the **ilike** operator in PostgreSQL, where patterns are case insensitive.

SQL expresses patterns by using the **like** comparison operator. Consider the query “Find the names of all departments whose building name includes the substring 'Watson'.” This query can be written as:

```
department(dept_name, building, budget)

select dept_name
from department
where building like '%Watson%';
```

For patterns to include the special pattern characters (that is, % and _), SQL allows the specification of an escape character. The escape character is used immediately before a special pattern character to indicate that the special pattern character is to be treated like a normal character. We define the escape character for a **like** comparison using the **escape** keyword. To illustrate, consider the following patterns, which use a backslash (\) as the escape character:

- **like** 'ab\%cd%' **escape** '\' matches all strings beginning with “ab%cd”.
- **like** 'ab\\cd%' **escape** '\' matches all strings beginning with “ab\cd”.

SQL allows us to search for mismatches instead of matches by using the **not like** comparison operator. Some implementations provide variants of the **like** operation that do not distinguish lower- and uppercase.

- Find the names of all instructors whose name includes the substring “dar”.

```
select name
from instructor
where name like '%dar%'
```

- Match the string “100%”

```
like '100 \% ' escape '\'
```

in that above we use backslash (\) as the escape character.

Attribute Specification in the Select Clause

The asterisk symbol “ * ” can be used in the select clause to denote “all attributes.” Thus, the use of *instructor.** in the select clause of the query:

```
select instructor.*
from instructor, teaches
where instructor.ID = teaches.ID;
```

*

Select *
from *instructor*

Select *instructor.**
from *instructor*

indicates that all attributes of *instructor* are to be selected. A select clause of the form **select *** indicates that all attributes of the result relation of the **from** clause are selected.

instructor(ID, name, dept_name, salary)

teaches(ID, course_id, sec_id, semester, year)

```
select instructor.*
from instructor, teaches
where instructor.ID = teaches.ID;
```

→

select * -

Ordering the Display of Tuples

SQL offers the user some control over the order in which tuples in a relation are displayed. The **order by** clause causes the tuples in the result of a query to appear in sorted order.

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors


```
select distinct name
from   instructor
order by name
```
- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
 - Example: **order by name desc**
- Can sort on multiple attributes
 - Example: **order by dept_name, name**

instructor(ID, name, dept_name, salary)

- List in alphabetic order all instructors in the Physics department.

```
select name
from instructor
where dept_name = 'Physics'
order by name;
```

```
select *
from instructor
order by salary desc, name asc;
```

- List the entire instructor relation in descending order of salary. If several instructors have the same salary, order them in ascending order by name.

Where Clause Predicates

- SQL includes a **between** comparison operator

instructor(ID, name, dept_name, salary)

SQL includes a **between** comparison operator to simplify **where** clauses that specify that a value be less than or equal to some value and greater than or equal to some other value. If we wish to find the names of instructors with salary amounts between \$90,000 and \$100,000, we can use the **between** comparison to write:

```
select name
from instructor
where salary between 90000 and 100000;
```

instead of:

```
select name
from instructor
where salary <= 100000 and salary >= 90000;
```

Similarly, we can use the **not between** comparison operator.

- Tuple comparison
 - **select** name, course_id
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');

instructor(ID, name, dept_name, salary)

teaches(ID, course_id, sec_id, semester, year)

- Find the instructor names and the courses they taught for all instructors in the Biology department who have taught some course.

```
select name, course_id
from instructor, teaches
where instructor.ID= teaches.ID and dept_name = 'Biology';
```

```
select name, course_id
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```

SQL permits us to use the notation $(v_1, v_2, \dots, v_n)$ to denote a tuple of arity n containing values $v_1, v_2, \dots, v_n$; the notation is called a *row constructor*. The comparison operators can be used on tuples, and the ordering is defined lexicographically. For example, $(a_1, a_2) \leq (b_1, b_2)$ is true if $a_1 \leq b_1$ **and** $a_2 \leq b_2$; similarly, the two tuples are equal if all their attributes are equal.

Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the mathematical set operations $\cup$, $\cap$, and $-$. We shall now construct queries involving the **union**, **intersect**, and **except** operations over two sets.

- Set operations **union**, **intersect**, and **except**
 - Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the
 - **union all**,
 - **intersect all**
 - **except all**.

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

- The set of all courses taught in the Fall 2009 semester.

```
select course_id
from section
where semester = 'Fall' and year= 2009;
```

- The set of all courses taught in the Spring 2010 semester.

```
select course_id
from section
where semester = 'Spring' and year= 2010;
```

Union Operation

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

- The set of all courses taught either in Fall 2009 or in Spring 2010,
or both.

 (select course_id
from section
where semester = 'Fall' and year= 2009)
 union
(select course_id
from section
where semester = 'Spring' and year= 2010);

 (select course_id
from section
where semester = 'Fall' and year= 2009)
 union all
(select course_id
from section
where semester = 'Spring' and year= 2010);

Intersect Operation

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

- The set of all courses taught in the Fall 2009 as well as in Spring
2010.

 (select course_id
from section
where semester = 'Fall' and year= 2009)
 intersect
(select course_id
from section
where semester = 'Spring' and year= 2010);

 (select course_id
from section
where semester = 'Fall' and year= 2009)
 intersect all
(select course_id
from section
where semester = 'Spring' and year= 2010);

Except Operation

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

- All courses taught in the Fall 2009 semester but not in the Spring 2010 semester.

```
(select course_id
from section
where semester = 'Fall' and year= 2009)
→ except
(select course_id
from section
where semester = 'Spring' and year= 2010);

(select course_id
from section
where semester = 'Fall' and year= 2009)
→ except all
(select course_id
from section
where semester = 'Spring' and year= 2010);
```

Suppose a tuple occurs m times in r and n times in s , then, it occurs:

- $m + n$ times in r **union all** s
- $\min(m, n)$ times in r **intersect all** s
- $\max(0, m - n)$ times in r **except all** s

$m > n$

$m = n$

$m < n$

Null Values

- It is possible for tuples to have a null value, denoted by **null**, for some of their attributes
- null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**
 - Example: $5 + \text{null}$ returns **null**
- The predicate **is null** can be used to check for null values.
 - Example: Find all instructors whose salary is null.


```
select name
from instructor
where salary is null
```
- The predicate **is not null** succeeds if the value on which it is applied is not null.
- SQL treats as **unknown** the result of any comparison involving a null value (other than predicates **is null** and **is not null**).
 - Example: $5 < \text{null}$ or $\text{null} <> \text{null}$ or $\text{null} = \text{null}$
- The predicate in a **where** clause can involve Boolean operations (**and**, **or**, **not**); thus the definitions of the Boolean operations need to be extended to deal with the value **unknown**.
 - and**: $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - or**: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
- Result of **where** clause predicate is treated as false if it evaluates to unknown

select sid
from Student
where pno is NULL

< >
=

Three-valued logic using the truth value unknown:

- OR**: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
- AND**: $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
- NOT**: $(\text{not unknown}) = \text{unknown}$

If the **where** clause predicate evaluates to either **false** or **unknown** for a tuple, that tuple is not added to the result.

SQL allows us to test whether the result of a comparison is unknown, rather than true or false, by using the clauses is unknown and is not unknown. For example,

```
select name
from instructor
where salary > 10000 is unknown;
```

“P is unknown” evaluates to **true** if predicate P evaluates to *unknown*

π

A	B	C
2	NULL	NULL
2	NULL	NULL

X

When a query uses the select distinct clause, duplicate tuples must be eliminated. For this purpose, when comparing values of corresponding attributes from two tuples, the values are treated as identical if either both are non-null and equal in value, or both are null. Thus, two copies of a tuple, such as {('A',null), ('A',null)}, are treated as being identical, even if some of the attributes have a null value. Using the **distinct** clause then retains only one copy of such identical tuples. Note that the treatment of null above is different from the way nulls are treated in predicates, where a comparison “null=null” would return unknown, rather than true.

The approach of treating tuples as identical if they have the same values for all attributes, even if some of the values are null, is also used for the set operations union, intersection, and except.

Aggregate Functions

Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value.

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value
min: minimum value
max: maximum value
sum: sum of values
count: number of values

Aggregate Functions Examples

instructor(ID, name, dept_name, salary)

- Find the average salary of instructors in the Computer Science department
 - select avg (salary)**
from instructor
where dept_name= 'Comp. Sci.';
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - select count (distinct ID)**
from teaches
where semester = 'Spring' and year = 2018;
- Find the number of tuples in the *course* relation
 - select count (*)**
from course;

Aggregate Functions –Group By

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

Figure 3.13 Tuples of the *instructor* relation, grouped by the *dept_name* attribute.

instructor(ID, name, dept_name, salary)

Find the average salary in each department.

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name;
```

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Figure 3.14 The result relation for the query “Find the average salary in each department”.

Figure 3.13 shows the tuples in the *instructor* relation grouped by the *dept_name* attribute, which is the first step in computing the query result. The specified aggregate is computed for each group, and the result of the query is shown in Figure 3.14.

In contrast, consider the query “Find the average salary of all instructors.” We write this query as follows:

```
select avg (salary)
from instructor;
```

In this case the **group by** clause has been omitted, so the entire relation is treated as a single group.

When an SQL query uses grouping, it is important to ensure that the only attributes that appear in the **select** statement without being aggregated are those that are present in the **group by** clause. In other words, any attribute that is not present in the **group by** clause may appear in the **select** clause only as an argument to an aggregate function, otherwise the query is treated as erroneous. For example, the following query is erroneous since *ID* does not appear in the **group by** clause, and yet it appears in the **select** clause without being aggregated:

```
/* erroneous query */
select dept_name, ID, avg (salary)
from instructor
group by dept_name;
```

In the preceding query, each instructor in a particular group (defined by *dept_name*) can have a different *ID*, and since only one tuple is output for each group, there is no unique way of choosing which *ID* value to output. As a result, such cases are disallowed by SQL.

Aggregate Functions –Having Clause

At times, it is useful to state a condition that applies to groups rather than to tuples. For example, we might be interested in only those departments where the average salary of the instructors is more than \$42,000. This condition does not apply to a single tuple; rather, it applies to each group constructed by the **group by** clause. To express such a query, we use the **having** clause of SQL. SQL applies predicates in the **having** clause after groups have been formed, so aggregate functions may be used in the **having** clause.

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg (salary) as avg_salary  
① from instructor  
② group by dept_name  
   having avg (salary) > 42000;
```

- Note: predicates in the **having** clause are applied after the formation of groups whereas predicates in the **where** clause are applied before forming groups

dept_name	avg_salary
Physics	91000
Elec. Eng.	80000
Finance	85000
Comp. Sci.	77333
Biology	72000
History	61000

Figure 3.16 The result relation for the query “Find the average salary of instructors in those departments where the average salary is more than \$42,000.”

Sequence of Operations

- As was the case for queries without aggregation, the **from** clause is first evaluated to get a relation.
- If a **where** clause is present, the predicate in the **where** clause is applied on the result relation of the **from** clause.
- Tuples satisfying the **where** predicate are then placed into groups by the **group by** clause if it is present. If the **group by** clause is absent, the entire set of tuples satisfying the **where** predicate is treated as being in one group.
- The **having** clause, if it is present, is applied to each group; the groups that do not satisfy the **having** clause predicate are removed.
- The **select** clause uses the remaining groups to generate tuples of the result of the query, applying the aggregate functions to get a single result tuple for each group.

In general, aggregate functions treat nulls according to the following rule: All aggregate functions except **count (*)** ignore null values in their input collection. As a result of null values being ignored, the collection of values may be empty. The **count** of an empty collection is defined to be 0, and all other aggregate operations return a value of null when applied on an empty collection. The effect of null values on some of the more complicated SQL constructs can be subtle.

Note: only one tuple is output for each group
Therefore for the following queries

```
select *  
from instructor  
group by dept name;
```

```
select dept name, salary  
from instructor  
group by dept name;
```

Compiler may output the following depending upon query language:

1. Error
2. First row of each group
3. Last row of each group

Since only one tuple is output for each group, there is no unique way of choosing which Tuple to output. As a result, such cases are disallowed by SQL.

```
select *  
from instructor  
group by dept name;
```

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
32343	El Said	History	60000
15151	Mozart	Music	40000
33456	Gold	Physics	87000

Group By multiple columns: Group by multiple column is say for example, **GROUP BY column1, column2**. This means to place all the rows with same values of both the columns **column1** and **column2** in one group.

Consider the below query:

```
SELECT SUBJECT, YEAR, Count(*)  
FROM Student  
GROUP BY SUBJECT, YEAR;
```

SUBJECT	YEAR	Count
English	1	3
English	2	2
Mathematics	1	2

SUBJECT	YEAR	NAME
English	1	Harsh
English	1	Pratik
English	1	Ramesh
English	2	Ashish
English	2	Suresh
Mathematics	1	Deepak
Mathematics	1	Sayan

SELECT * FROM t1, t2 JOIN t3

$$(R_1 \times R_2) \bowtie R_3$$

JOIN has higher precedence than the comma operator (,), so the join expression t1, t2 JOIN t3 is interpreted as (t1, (t2 JOIN t3)), not as ((t1, t2) JOIN t3). This affects statements that use an ON clause because that clause can refer only to columns in the operands of the join, and the precedence affects interpretation of what those operands are.

From <<https://dev.mysql.com/doc/refman/8.0/en/join.html>>

SELECT * FROM (t1, t2) JOIN t3

$$(R_1 \times R_2) \bowtie R_3$$

$$R_1 \times R_2 \bowtie R_3$$

$$R_1, R_2 \text{ join } R_3$$

$$R_1 \times (R_2 \bowtie R_3)$$

The general form of an SQL query with these extensions is:

```

SELECT    [ DISTINCT ] select-list
FROM      from-list
WHERE     qualification
GROUP BY  grouping-list
HAVING    group-qualification
ORDER BY  order-item-list

```

instructor(ID, name, dept_name, salary)

```

select *
from instructor
order by salary desc, name asc;

```

- List the entire instructor relation in descending order of salary. If several instructors have the same salary, order them in ascending order by name.

